



Composite e Flyweight

Padrões de Projeto - Aula #04

Prof. Dr. Carlos Melo

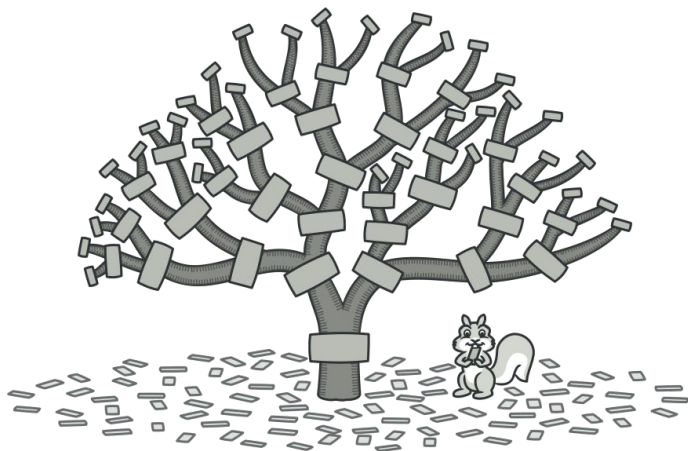
 carlos.melo@upe.br



Padrões Estruturais (parte 3)

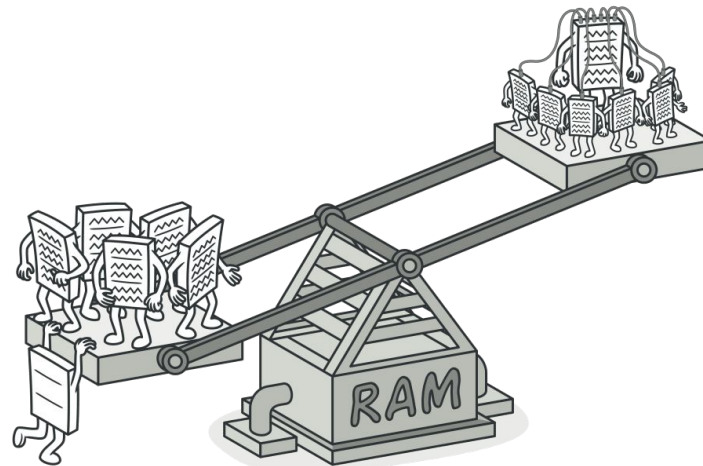
Composite

Tratar um único objeto ou uma árvore de objetos exatamente do mesmo jeito



Flyweight

Milhões de objetos que, na verdade, compartilham quase todos os dados

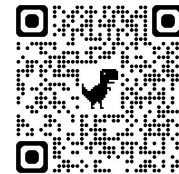


- Ambos os padrões focam em problemas de escala: um sobre profundidade da estrutura e o outro o número de objetos



Objetivos de aprendizagem

- Identificar quando um problema é recursivo por natureza e por que **isinstance** indica a falta de um Composite.
- Decidir entre transparência e segurança ao projetar a interface do Componente no Composite.
- Separar estado **intrínseco** de estado **extrínseco** e medir a economia de memória de um Flyweight em Python.
- Entender o porque do Flyweight exigir imutabilidade e ser o padrão mais fácil de aplicar errado.



Composite - O Problema

- Um novo problema com nosso sistema de armazenamento de imagens extraterrestres
- Precisamos responder às seguintes perguntas:
 - Qual o tamanho total de 2026-08-02/?
 - Quantos arquivos existem sob acervo/?
 - Onde está andromeda.tiff?

```
1 acervo/
2 | 2026-08-01/
3 | | hubble_001.fits      450 MB
4 | | hubble_002.fits      450 MB
5 | | calibracao/
6 | | | dark_frame.fits    80 MB
7 | | | flat_field.fits    80 MB
8 | | 2026-08-02/
9 | | | hubble_003.fits    450 MB
10 | | | processadas/
11 | | | | mosaico/
12 | | | | | andromeda.tiff 1200 MB
13 | | | | | hubble_003_lut.png 2 MB
14 | | README.md           0.01 MB
```





Uma solução

- Funciona, mas não necessariamente é a melhor solução
- Note que o cliente é obrigado a saber que existem dois tipos diferentes para fazer uma pergunta

```
1 class Arquivo:
2     def __init__(self, nome: str, tamanho: float):
3         self.nome = nome
4         self.tamanho = tamanho
5
6 class Pasta:
7     def __init__(self, nome: str):
8         self.nome = nome
9         self.filhos = []
```

```
1 def tamanho_total(item) -> float:
2     if isinstance(item, Arquivo):
3         return item.tamanho
4     elif isinstance(item, Pasta):
5         total = 0.0
6         for filho in item.filhos:
7             total += tamanho_total(filho)
8         return total
9     raise TypeError(f"tipo desconhecido: {type(item)}")
```

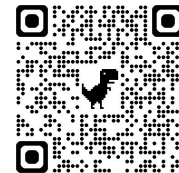


Composite - O Problema

- O acervo cresce e chegam novos requisitos

Requisito	Consequência
Contar arquivos	Nova funcionalidade, mesma cadeia de instance
Buscar por nome	Nova funcionalidade, mesma cadeia de instance
Listar em árvore	Nova funcionalidade, mesma cadeia de instance
Suporte a atalhos	Editar todas as funções existentes

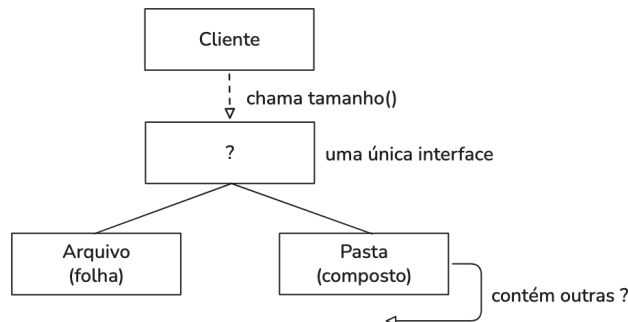
- 4 operações x 3 tipos = 12 ramos de **instance** espalhados pelo código
- **if/elif** ad eternum
 - Violamos o princípio Aberto/Fechado (● do SOLID)
- Por que o cliente precisa saber se está lidando com um arquivo ou com uma pasta, se a pergunta que ele faz é a mesma?



Composite - Ideia

- O cliente deveria poder escrever apenas:

```
1 print(item.tamanho()) # sem saber o que é um item (pasta/arquivo)
```



- A folha responde com o próprio valor
- O composto (contêiner) responde perguntando aos filhos e somando e cada filho pode ser folha ou composto, indefinidamente
- A recursão sai do cliente e entra na estrutura das classes
- O Composite foi criado justamente para esse tipo de problema



Composite - Ideia

*"Compor objetos em estruturas de árvore para representar hierarquias **parte-todo**. O Composite permite que os clientes tratem objetos individuais e composições de objetos de maneira uniforme."*

- Padrão estrutural
 - Parte-todo: é uma árvore e um falho tem a mesma natureza da árvore
 - Uniforme: o cliente não distingue folha de composto (contêiner)
- A recursão sai do cliente e entra na estrutura das classes
- O cliente não sabe com o que está falando (esse é o objetivo)



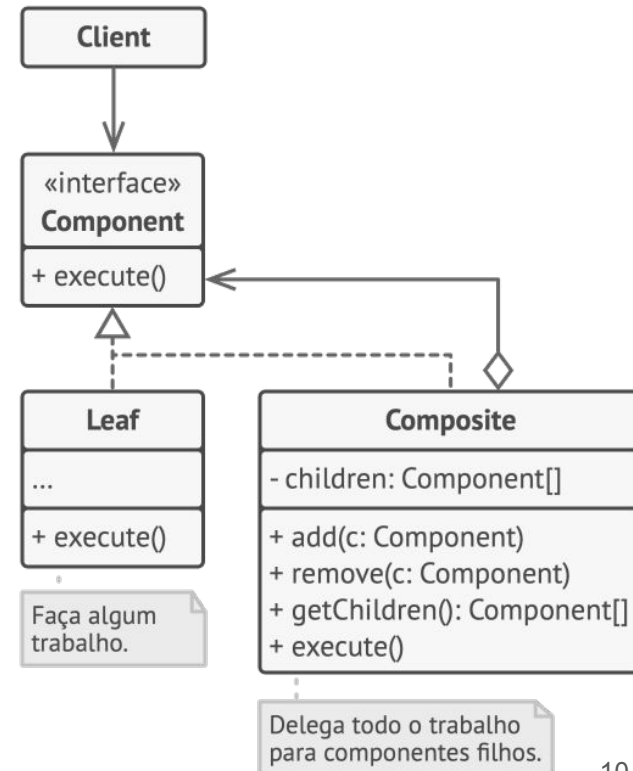
Composite - Onde é utilizado?

Domínio	Folha	Composto
Sistema de arquivos	Arquivo	diretório
HTML/DOM	texto, 	<div> , <body>
GUI	botão, label	painel, janela
Organograma	Funcionário	Departamento

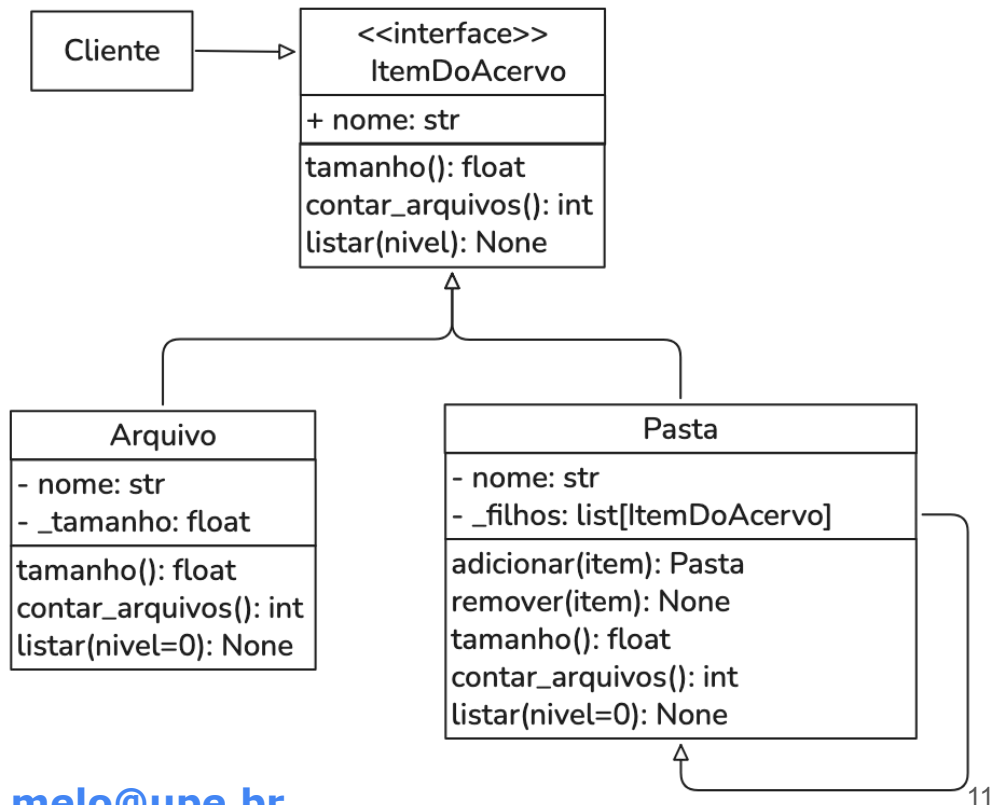
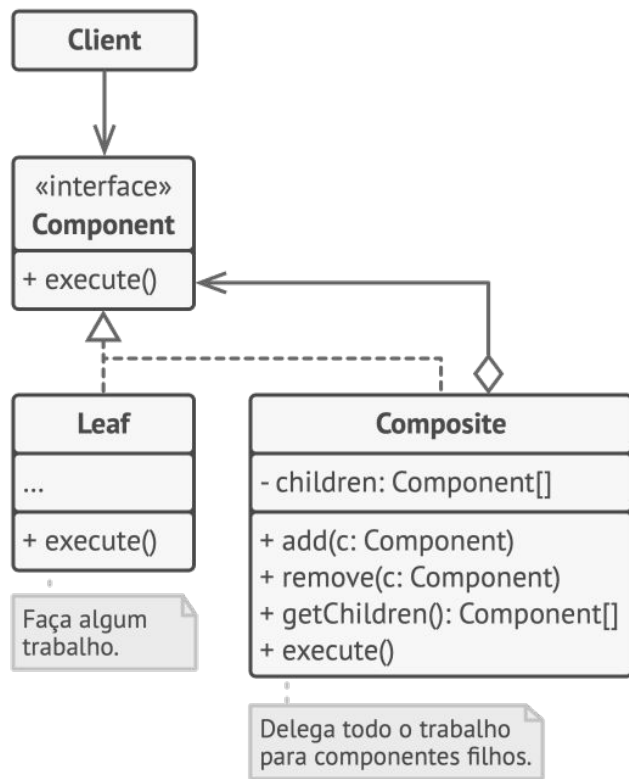
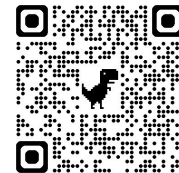


Composite - Estrutura

1. A interface Component descreve as operações que são comuns tanto para elementos simples como para elementos complexos da árvore
2. A Leaf é um elemento básico de uma árvore que não tem subelementos
3. O Composite é o elemento que tem subelementos: folhas ou outros contêineres
4. O Client trabalha com todos os elementos através da interface Component.



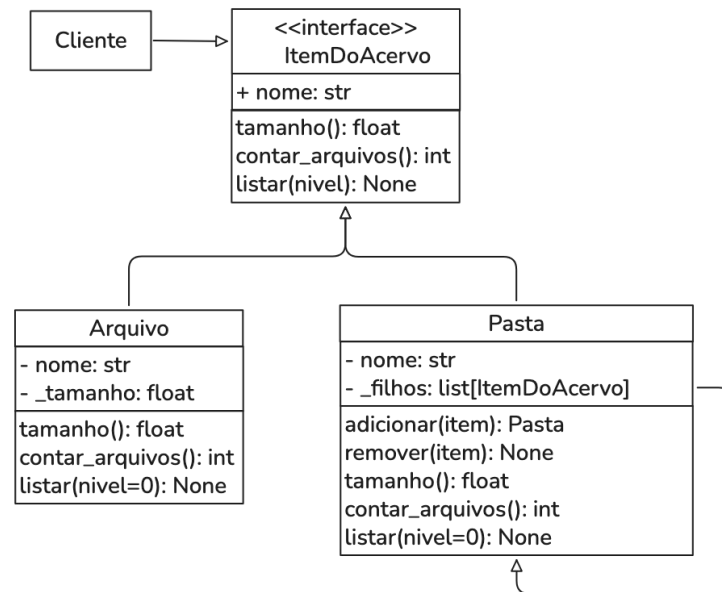
Aplicando ao nosso domínio

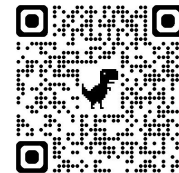




Aplicando ao nosso domínio

- Component (ItemDoAcervo) declara todas as operações comuns.
 - É a única coisa que o cliente conhece.
- Leaf (Arquivo) implementa as operações fazendo o trabalho de fato.
 - Não tem filhos.
- Composite (Pasta) guarda uma lista de ItemDoAcervo e implementa cada operação delegando aos filhos e agregando o resultado.





Aplicando ao nosso domínio

```
1 from abc import ABC, abstractmethod
2
3 class ItemDoAcervo(ABC):
4     def __init__(self, nome: str):
5         self.nome = nome
6
7     @abstractmethod
8     def tamanho(self) -> float:
9
10    @abstractmethod
11    def contar_arquivos(self) -> int: ...
12
13    @abstractmethod
14    def listar(self, nivel: int = 0) -> None: ...
```

- Três operações, uma interface.
- Por que não colocar `adicionar()` e `remover()` na interface?



Aplicando ao nosso domínio

```
1 class Arquivo(ItemDoAcervo):
2     def __init__(self, nome: str, tamanho_mb: float):
3         super().__init__(nome)
4         self._tamanho = tamanho_mb
5
6     def tamanho(self) -> float:
7         return self._tamanho # caso base
8
9     def contar_arquivos(self) -> int:
10        return 1 # 1 arquivo
11
12    def listar(self, nivel: int = 0) -> None:
13        print(f"{' ' * nivel} {self.nome} ({self._tamanho:.2f} MB)")
```

- A folha é onde a recursão para. tamanho() devolve um valor, contar_arquivos() devolve 1.
- Três métodos, nenhum if, nenhuma menção a Pasta.
- A folha não sabe que faz parte de uma árvore.

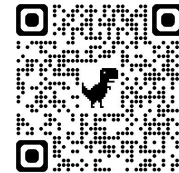


Aplicando ao nosso domínio

```
1 class Pasta(ItemDoAcervo):
2     def __init__(self, nome: str):
3         super().__init__(nome)
4         self._filhos: list[ItemDoAcervo] = []
5
6     def adicionar(self, item: ItemDoAcervo) -> "Pasta":
7         self._filhos.append(item)
8         return self # permite encadear
9
10    def remover(self, item: ItemDoAcervo) -> None:
11        self._filhos.remove(item)
12
13    def tamanho(self) -> float:
14        return sum(filho.tamanho() for filho in self._filhos)
15
16    def contar_arquivos(self) -> int:
17        return sum(filho.contar_arquivos() for filho in self._filhos)
18
19    def listar(self, nivel: int = 0) -> None:
20        print(f'{' ' * nivel} {self.nome}/ ({self.tamanho():.2f} MB)")
21        for filho in self._filhos:
22            filho.listar(nivel + 1) # recursão
```

- As três operações têm a mesma forma: percorrer os filhos, agregar. Nenhuma sabe o tipo do filho.
- Três métodos, nenhum if, nenhuma menção a Pasta.
- A folha não sabe que faz parte de uma árvore.

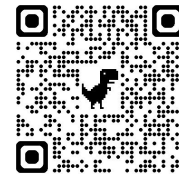
Composite - Testando



```
1 acervo = Pasta("acervo")
2
3 dia_01 = Pasta("2026-08-01")
4 dia_01.adicionar(Arquivo("hubble_001.fits", 450))
5 dia_01.adicionar(Arquivo("hubble_002.fits", 450))
6
7 calib = Pasta("calibracao")
8 calib.adicionar(Arquivo("dark_frame.fits", 80))
9 calib.adicionar(Arquivo("flat_field.fits", 80))
10 dia_01.adicionar(calib)
11
12 dia_02 = Pasta("2026-08-02")
13 dia_02.adicionar(Arquivo("hubble_003.fits", 450))
14
15 mosaico = Pasta("mosaico").adicionar(Arquivo("andromeda.tiff", 1200))
16 proc = Pasta("processadas").adicionar(mosaico)
17 proc.adicionar(Arquivo("hubble_003_lut.png", 2))
18 dia_02.adicionar(proc)
19
20 acervo.adicionar(dia_01).adicionar(dia_02)
21 acervo.adicionar(Arquivo("README.md", 0.01))
```

```
1 acervo.listar()
2 print(f"\nTotal: {acervo.tamanho():.2f} MB em {acervo.contar_arquivos()} arquivos")
3 print(f"Só 2026-08-02: {dia_02.tamanho():.2f} MB")
```

Composite - Testando



```
1 acervo = Pasta("acervo")
2
3 dia_01 = Pasta("2026-08-01")
4 dia_01.adicionar(Arquivo("hubble_001.fits", 450))
5 dia_01.adicionar(Arquivo("hubble_002.fits", 450))
6
7 calib = Pasta("calibracao")
8 calib.adicionar(Arquivo("dark_frame.fits", 80))
9 calib.adicionar(Arquivo("flat_field.fits", 80))
10 dia_01.adicionar(calib)
11
12 dia_02 = Pasta("2026-08-02")
13 dia_02.adicionar(Arquivo("hubble_003.fits", 450))
14
15 mosaico = Pasta("mosaico").adicionar(Arquivo("andromeda.tiff", 1200))
16 proc = Pasta("processadas").adicionar(mosaico)
17 proc.adicionar(Arquivo("hubble_003_lut.png", 2))
18 dia_02.adicionar(proc)
19
20 acervo.adicionar(dia_01).adicionar(dia_02)
21 acervo.adicionar(Arquivo("README.md", 0.01))
```

```
1 acervo/ (2712.01 MB)
2 2026-08-01/ (1060.00 MB)
3   hubble_001.fits (450.00 MB)
4   hubble_002.fits (450.00 MB)
5   calibracao/ (160.00 MB)
6     dark_frame.fits (80.00 MB)
7     flat_field.fits (80.00 MB)
8 2026-08-02/ (1652.00 MB)
9   hubble_003.fits (450.00 MB)
10  processadas/ (1202.00 MB)
11    mosaico/ (1200.00 MB)
12      andromeda.tiff (1200.00 MB)
13      hubble_003_lut.png (2.00 MB)
14  README.md (0.01 MB)
15
16 Total: 2712.01 MB em 8 arquivos
17 Só 2026-08-02: 1652.00 MB
```

```
1 acervo.listar()
2 print(f"\nTotal: {acervo.tamanho():.2f} MB em {acervo.contar_arquivos()} arquivos")
3 print(f"Só 2026-08-02: {dia_02.tamanho():.2f} MB")
```



Exercício

- Implemente o método **buscar()** que permite encontrar um arquivo ou uma pasta.
- Implemente o método **caminho()** que retorna o path até o arquivo ou pasta que buscamos.



Prós e Contras do Composite

Prós

- Estruturas de árvore de profundidade arbitrária sem uma linha de código sobre profundidade.
- O cliente trata folha e contêiner igualmente
- A recursão fica encapsulada na estrutura, não espalhada pelos clientes.
- Respeita o **O** do **SOLID**

Contras

- Se as folhas são muito diferentes entre si, a interface comum fica artificialmente genérica.
- Difícil restringir os tipos de filhos de um composite:
 - nada impede `Pasta("x").adicionar(Pasta("x"))` criando um ciclo
- Operações agregadas podem ser caras: `acervo.tamanho()` percorre a árvore inteira a cada chamada
- Debug de recursão é desconfortável (o stack fica cheio).

Não use:

- Se a estrutura tem profundidade fixa e conhecida (exatamente 2 níveis, por exemplo), duas classes simples são mais legíveis.



Flyweight - O Problema

- O acervo virou um catálogo de estrelas. Cada imagem do Hubble tem milhões de objetos identificados, e o front-end precisa renderizar todos:

```
1 class Estrela:
2     def __init__(self, x, y, tipo_espectral, cor_rgb, textura, curva_luminosidade, catalogo_ref):
3         self.x = x
4         self.y = y
5         self.tipo_espectral = tipo_espectral
6         self.cor_rgb = cor_rgb
7         self.textura = textura
8         self.curva_luminosidade = curva_luminosidade
9         self.catalogo_ref = catalogo_ref
```

```
1 estrelas = [Estrela(x, y, "G", (255,244,234), textura_g, curva_g, "MK")
2             for x, y in posicoes]
```

- Navegador travou.
- Existem 7 tipos espectrais de estrelas e 2 milhões delas. O que dá aproximadamente 40GB.



Flyweight - Ideia

- Reduzir de 40 GB para 112 MB ao reaproveitar informação em comum entre os objetos.

Campo	Tamanho	Varia por estrela?
x, y	56 B	Sim
tipo_espectral	~50 B	7 valores possíveis
cor_rgb	~72 B	Derivado do tipo
textura	12 KB	
curva_luminosidade	8 KB	
catalogo_ref	~60 B	Sempre MK (Morgan-Keenan)



Flyweight - Estado intrínseco x extrínseco

```
1 # Tudo junto, 2_000_000 de vezes
2 class Estrela:
3     def __init__(self, x, y, tipo, cor, textura, curva, catalogo): ...
4
5 # 0 que se repete sai de dentro
6 class TipoEstrela: # 7 instâncias
7     def __init__(self, tipo, cor, textura, curva): ...
8     def desenhar(self, tela, x, y): ... # ➡ x,y vêm de fora
9
10 class EstrelaNaImagem: # 2_000_000 instâncias minúsculas
11     def __init__(self, x, y, tipo: TipoEstrela): ...
```

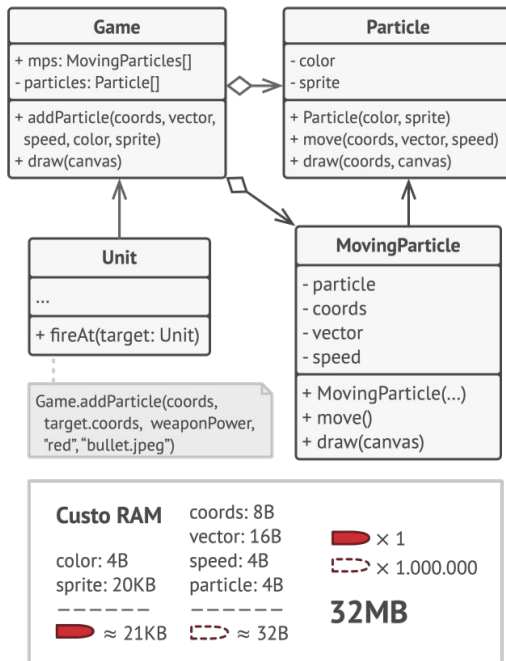
- A operação **desenhar()** muda de assinatura: o estado extrínseco entra como argumento.
- Se remover o campo faz dois objetos ficarem idênticos, ele era extrínseco.

Flyweight - Estrutura



"Usar compartilhamento para suportar eficientemente grandes quantidades de objetos de granularidade fina."

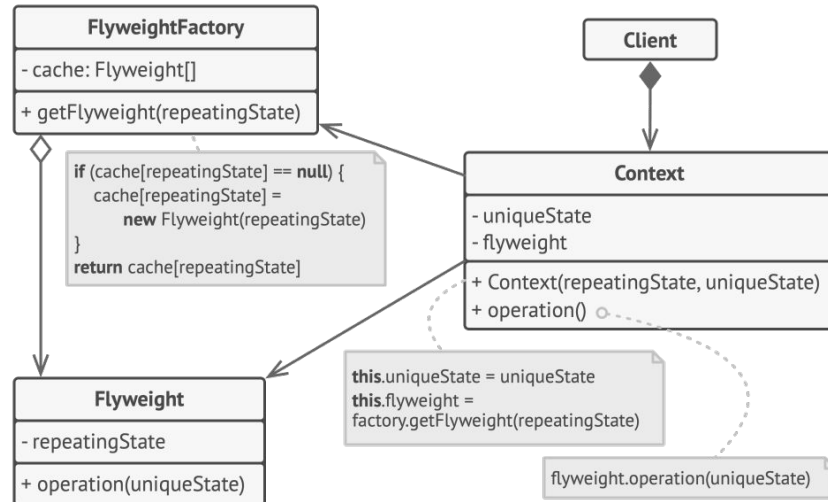
- É o único dos 22 cuja motivação é exclusivamente desempenho
 - Os outros tratam de acoplamento e flexibilidade.
- Não aplique Flyweight antes de medir.
 - Ele piora a legibilidade em troca de memória.



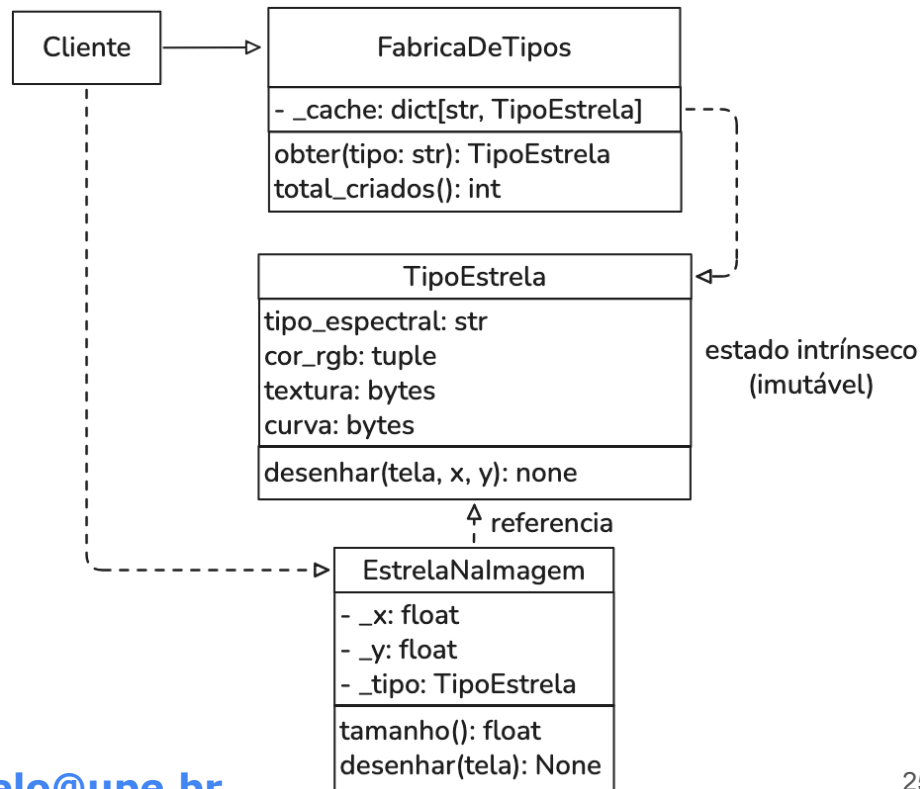
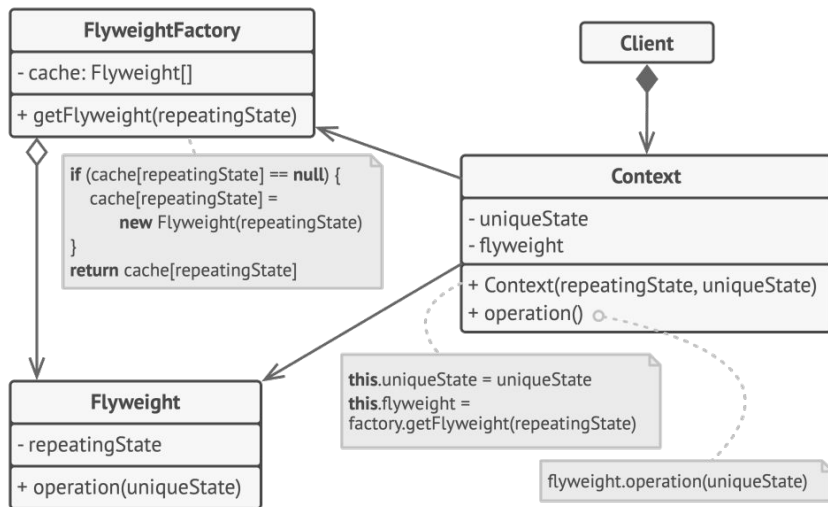
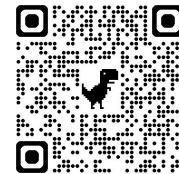


Flyweight - Estrutura

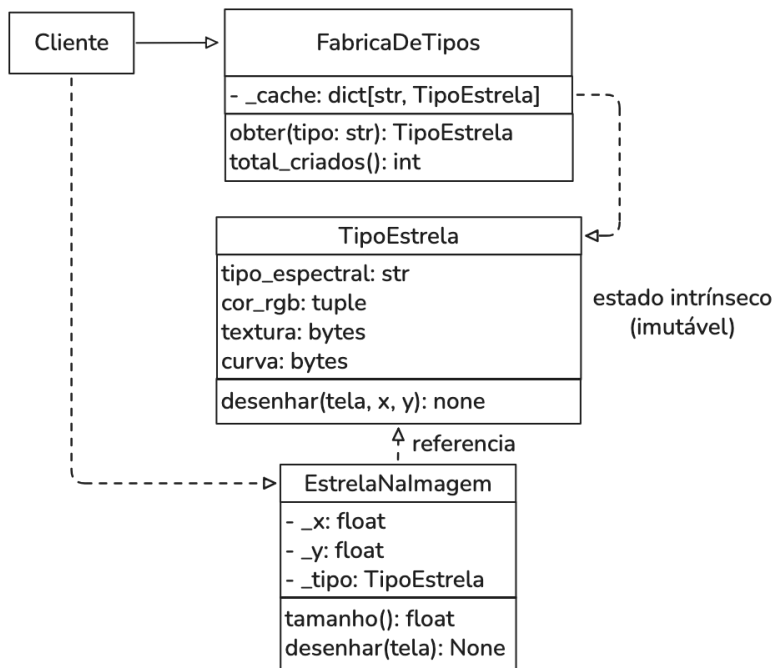
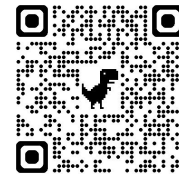
1. Flyweight contém a porção do estado do objeto original que pode ser compartilhada
2. Context contém o estado extrínseco, único para todos os objetos originais
3. O Client calcula ou armazena o estado extrínseco dos flyweights
4. A FlyweightFactory gerencia um conjunto de flyweights existentes, ou seja, o Client não precisa criar flyweights diretamente



Aplicando ao nosso domínio

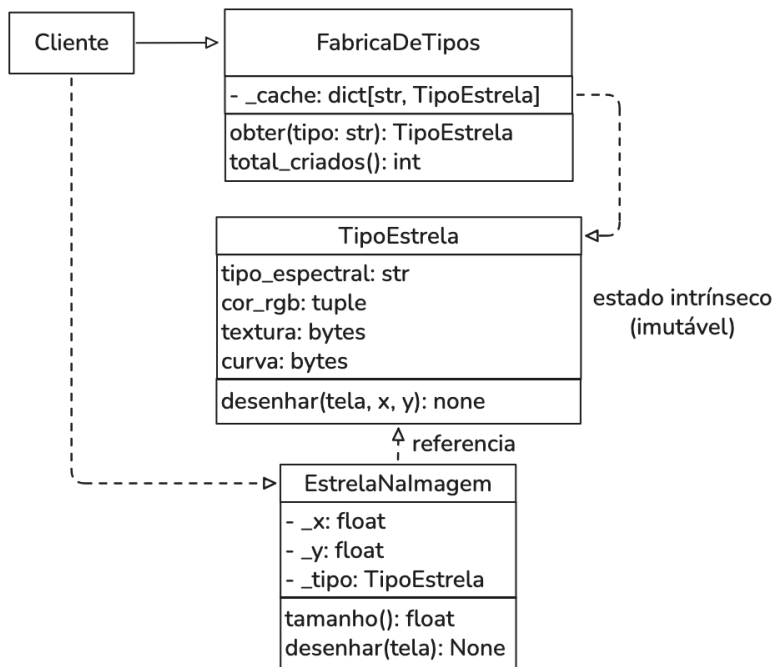
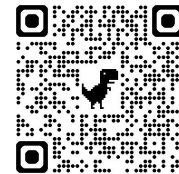


Aplicando ao nosso domínio



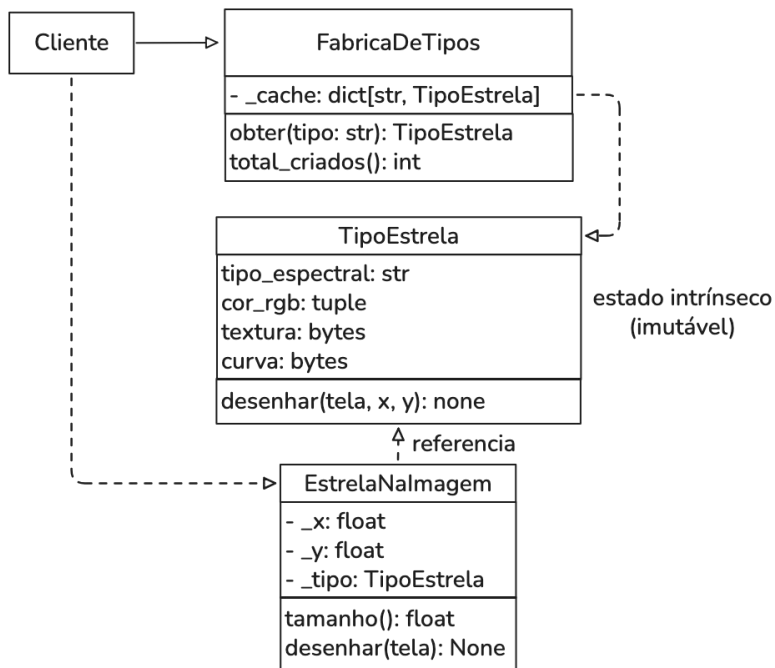
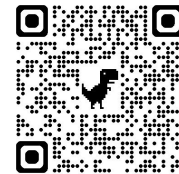
```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True, slots=True) # imutável
4 class TipoEstrela:
5     tipo_espectral: str
6     cor_rgb: tuple[int, int, int]
7     textura: bytes
8     curva: bytes
9
10     def desenhar(self, tela: str, x: float, y: float) -> None:
11         print(f"[{tela}] {self.tipo_espectral} rgb{self.cor_rgb} "
12               f"em ({x:.1f}, {y:.1f})")
```

Aplicando ao nosso domínio



```
1 class FabricaDeTipos:
2     _cache: dict[str, TipoEstrela] = {}
3
4     # dados do sistema Morgan-Keenan
5     _CATALOGO = {
6         "O": (155, 176, 255), "B": (170, 191, 255), "A": (202, 215, 255),
7         "F": (248, 247, 255), "G": (255, 244, 234), "K": (255, 210, 161),
8         "M": (255, 204, 111),
9     }
10
11     @classmethod
12     def obter(cls, tipo: str) -> TipoEstrela:
13         if tipo not in cls._cache:
14             print(f"[FABRICA] criando flyweight '{tipo}' (1ª vez)")
15             cls._cache[tipo] = TipoEstrela(
16                 tipo_espectral=tipo,
17                 cor_rgb=cls._CATALOGO[tipo],
18                 textura=b"\x00" * 12_288,
19                 curva=b"\x00" * 8_192,
20             )
21         return cls._cache[tipo]
22
23     @classmethod
24     def total_criados(cls) -> int:
25         return len(cls._cache)
```

Aplicando ao nosso domínio



```
1 class EstrelaNalImagem:
2     __slots__ = ("_x", "_y", "_tipo")
3
4     def __init__(self, x: float, y: float, tipo: TipoEstrela):
5         self._x, self._y, self._tipo = x, y, tipo
6
7     def desenhar(self, tela: str) -> None:
8         self._tipo.desenhar(tela, self._x, self._y)
```



Prós e Contras do Flyweight

Prós

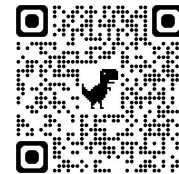
- Economia de RAM que pode ser de duas ordens de magnitude.
- Menos objetos criados, permite menos pressão no garbage collector.
- O cliente quase não muda: só passa a pedir à factory.

Contras

- Se o flyweight não é imutável, alterar um afeta todos que o compartilham.
 - Use **frozen=True**. Sempre.
- Flyweight não é cache.
 - Flyweight garante unicidade por identidade
 - cache guarda resultado de operação custosa e pode ser invalidado
 - Um flyweight nunca é descartado.

Não use:

- Se você tem 500 objetos, o Flyweight só piora o código.
- O padrão paga a partir de centenas de milhares de objetos.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br